1. Java Implementation

Marking Smells in the Original Code

```java
class UniversityStaff {
    // SMELL 1: Incorrect/Unclear naming of variables
    String n; // name
    String i; // id
    String d; // department
    int y;    // years of experience
    int c;    // certifications
    String k; // designation

    // SMELL 2: Long Parameter List (Needs a Parameter Object)
    public UniversityStaff(String n, String i, String d, int y, int c, String k) {
        this.n = n;
        this.i = i;
        this.d = d;
        this.y = y;
        this.c = c;
        this.k = k;
    }

    public String performDuty() {
        return n + " can perform academic duty"; // n is an unclear variable name
    }

    public int calcSalary() {
        // SMELL 3: Duplicated Code
        int baseSalary = 60000 + (y * 2);
        int salary = baseSalary;
        if (k.equals("Assistant Professor")) {
            salary = baseSalary + (c * 500);
        } else if (k.equals("Associate Professor")) {
            salary = baseSalary + (c * 800);
        }
        return salary;
    }

    public String displaySalary() {
        // SMELL 3: Duplicated Code (Calculation repeated unnecessarily)
        int baseSalary = 60000 + (y * 2);
        int salary = baseSalary;
        if (k.equals("Assistant Professor")) {
            salary = baseSalary + (c * 500);
        } else if (k.equals("Associate Professor")) {
            salary = baseSalary + (c * 800);
        }
        return n + " earns " + calcSalary();
    }
}

class LabAssistant extends UniversityStaff {
    String sh; // shift timing
    public LabAssistant(String n, String i, String d, int y, int c, String k,
    String shift) {
        super(n, i, d, y, c, k);
        this.sh = shift;
    }
```

```
}

class Professor extends UniversityStaff {
    String f; // specialization field
    public Professor(String n, String i, String d, int y, int c, String k, String field) {
        super(n, i, d, y, c, k);
        this.f = field;
    }

    public void evaluateResearchRisk(Student student) {
        String ageRisk;
        // SMELL 4: Feature Envy (Uses Student's data heavily, should be in Student)
        // SMELL 5: Long conditional
        if (student.age < 18 || student.age > 60) {
            ageRisk = "High";
        } else {
            ageRisk = "Low";
        }
    }
}

class Student {
    String name;
    int age;
    String records;

    public Student(String name, int age, String records) {
        this.name = name;
        this.age = age;
        this.records = records;
    }

    // SMELL 6: Long Method
    public boolean enrollmentEligibility() {
        boolean ageOk;
        if (age > 17 && age < 25) {
            ageOk = true;
        } else {
            ageOk = false;
        }

        boolean recordOk;
        if (records.equals("discipline_issue")) {
            recordOk = true;
        } else {
            recordOk = false;
        }
        return ageOk && recordOk;
    }
}
```

Refactored Code

```
// Refactoring 2: Introduce Parameter Object to fix Long Parameter List
class StaffProfile {
    // Refactoring 1: Correct and meaningful variable names
```

```
    private String name;
    private String id;
    private String department;
    private int yearsOfExperience;
    private int certifications;
    private String designation;

    public StaffProfile(String name, String id, String department,
                        int yearsOfExperience, int certifications, String
    designation) {
        this.name = name;
        this.id = id;
        this.department = department;
        this.yearsOfExperience = yearsOfExperience;
        this.certifications = certifications;
        this.designation = designation;
    }

    public String getName() { return name; }
    public String getId() { return id; }
    public String getDepartment() { return department; }
    public int getYearsOfExperience() { return yearsOfExperience; }
    public int getCertifications() { return certifications; }
    public String getDesignation() { return designation; }
}

class UniversityStaff {
    protected StaffProfile profile;

    public UniversityStaff(StaffProfile profile) {
        this.profile = profile;
    }

    public String performDuty() {
        return profile.getName() + " can perform academic duty";
    }

    // Refactoring 3: Extracted duplicated salary calculation into a separate
    method
    private int computeSalaryAmount() {
        int baseSalary = 60000 + (profile.getYearsOfExperience() * 2);
        int salary = baseSalary;

        if ("Assistant Professor".equals(profile.getDesignation())) {
            salary = baseSalary + (profile.getCertifications() * 500);
        } else if ("Associate Professor".equals(profile.getDesignation())) {
            salary = baseSalary + (profile.getCertifications() * 800);
        }
        return salary;
    }

    // Both methods now call the extracted method
    public int calcSalary() {
        return computeSalaryAmount();
    }

    public String displaySalary() {
        return profile.getName() + " earns " + computeSalaryAmount();
    }
```

```
}

class LabAssistant extends UniversityStaff {
    private String shiftTiming;

    public LabAssistant(StaffProfile profile, String shiftTiming) {
        super(profile);
        this.shiftTiming = shiftTiming;
    }
}

class Professor extends UniversityStaff {
    private String specializationField;

    public Professor(StaffProfile profile, String specializationField) {
        super(profile);
        this.specializationField = specializationField;
    }
    // evaluateResearchRisk has been moved to Student class (Refactoring 4)
}

class Student {
    private String name;
    private int age;
    private String records;

    public Student(String name, int age, String records) {
        this.name = name;
        this.age = age;
        this.records = records;
    }

    public int getAge() { return age; }

    // Refactoring 4: Move Method (from Professor) and Refactoring 5: Extract
    Method
    public String evaluateResearchRisk() {
        return isAgeRisk() ? "High" : "Low";
    }

    private boolean isAgeRisk() {
        return age < 18 || age > 60;
    }

    // Refactoring 6: Extract Methods used to simplify long method logic
    public boolean enrollmentEligibility() {
        return isAgeEligible() && isRecordEligible();
    }

    private boolean isAgeEligible() {
        return age > 17 && age < 25;
    }

    private boolean isRecordEligible() {
        return "discipline_issue".equals(records);
    }
}
```

2. Python Implementation

Marking Smells in the Original Code

```python
class UniversityStaff:
    # SMELL 2: Long Parameter List
    def __init__(self, n, i, d, y, c, k, m):
        # SMELL 1: Incorrect/Unclear naming of variables
        self.n = n  # name
        self.i = i  # id
        self.d = d  # department
        self.y = y  # years of experience
        self.c = c  # certifications
        self.k = k  # designation

    def perform_duty(self):
        return f"{self.n} can perform academic duty"

    def calc_salary(self):
        # SMELL 3: Duplicated Code
        base_salary = 60000 + (self.y * 2)
        if self.k == "Assistant Professor":
            salary = base_salary + (self.c * 500)
        elif self.k == "Associate Professor":
            salary = base_salary + (self.c * 800)
        else:
            salary = base_salary
        return salary

    def display_salary(self):
        # SMELL 3: Duplicated Code (Calculation repeated unnecessarily)
        base_salary = 60000 + (self.y * 2)
        if self.k == "Assistant Professor":
            salary = base_salary + (self.c * 500)
        elif self.k == "Associate Professor":
            salary = base_salary + (self.c * 800)
        else:
            salary = base_salary

        return f"{self.n} earns {self.calc_salary()}"


class Professor(UniversityStaff):
    def __init__(self, n, i, d, y, c, k, field):
        super().__init__(n, i, d, y, c, k, None)
        self.f = field  # specialization field

    def evaluate_research_risk(self, student):
        # SMELL 4: Feature Envy (Uses Student's data heavily, should be in Student
    )
        # SMELL 5: Long conditional
        if student.age < 18 or student.age > 60:
            age_risk = "High"
        else:
            age_risk = "Low"


class LabAssistant(UniversityStaff):
    def __init__(self, n, i, d, y, c, k, shift):
```

```
        super().__init__(n, i, d, y, c, k, None)
        self.sh = shift  # shift timing


class Student:
    def __init__(self, name, age, records):
        self.name = name
        self.age = age
        self.records = records

    # SMELL 6: Long Method (Use Extract Method)
    def enrollment_eligibility(self):
        if self.age > 17 and self.age < 25:
            age_ok = True
        else:
            age_ok = False

        if self.records in ["discipline_issue"]:
            record_ok = True
        else:
            record_ok = False

        return (age_ok and record_ok)
```

Refactored Code

```
from dataclasses import dataclass

# Refactoring 2: Introduce Parameter Object (Dataclass) to fix Long Parameter List
@dataclass
class StaffProfile:
    # Refactoring 1: Correct and meaningful variable names
    name: str
    staff_id: str
    department: str
    years_of_experience: int
    certifications: int
    designation: str


class UniversityStaff:
    def __init__(self, profile: StaffProfile):
        self.profile = profile

    def perform_duty(self):
        return f"{self.profile.name} can perform academic duty"

    # Refactoring 3: Extracted duplicated salary calculation into a separate
    method
    def _compute_salary_amount(self):
        base_salary = 60000 + (self.profile.years_of_experience * 2)
        salary = base_salary

        if self.profile.designation == "Assistant Professor":
            salary = base_salary + (self.profile.certifications * 500)
        elif self.profile.designation == "Associate Professor":
            salary = base_salary + (self.profile.certifications * 800)

```

```
        return salary

    # Both methods now call the extracted method
    def calc_salary(self):
        return self._compute_salary_amount()

    def display_salary(self):
        return f"{self.profile.name} earns {self._compute_salary_amount()}"


class Professor(UniversityStaff):
    def __init__(self, profile: StaffProfile, specialization_field: str):
        super().__init__(profile)
        self.specialization_field = specialization_field
    # evaluate_research_risk has been moved to Student class (Refactoring 4)


class LabAssistant(UniversityStaff):
    def __init__(self, profile: StaffProfile, shift_timing: str):
        super().__init__(profile)
        self.shift_timing = shift_timing


class Student:
    def __init__(self, name: str, age: int, records: str):
        self.name = name
        self.age = age
        self.records = records

    # Refactoring 4: Move Method (from Professor) and Refactoring 5: Extract
    Method
    def evaluate_research_risk(self):
        return "High" if self._is_age_risk() else "Low"

    def _is_age_risk(self):
        return self.age < 18 or self.age > 60

    # Refactoring 6: Extract Methods to simplify logic
    def enrollment_eligibility(self):
        return self._is_age_eligible() and self._is_record_eligible()

    def _is_age_eligible(self):
        return 17 < self.age < 25

    def _is_record_eligible(self):
        return self.records == "discipline_issue"
```